

Git Commands Practice

A) Configure & Environment

1) Show the version of Git installed on your system.

Command(s) — Answer:

```
git --version
```

Expected effect / output — Answer:

```
git version 2.51.2.windows.1
```

2) Set your global name to “Your name” and email to “your bcit email address”.

Command(s) — Answer:

```
git config --global user.name "Nathan"
```

```
git config --global user.email "nsiu7@my.bcit.ca"
```

3) Set the default branch name for new repositories to “main”.

Command(s) — Answer:

```
git config --global init.defaultBranch main
```

4) (Windows only) Configure Git to automatically handle CRLF line endings.

Command(s) — Answer:

```
git config --global core.autocrlf true
```

5) Display your current global configuration and where each setting comes from.

Command(s) — Answer:

```
git config --list --show-origin
```

Expected effect / output — Answer:

```
file:C:/Program Files/Git/etc/gitconfig diff.astextplain.textconv=astextplain
file:C:/Program Files/Git/etc/gitconfig filter.lfs.clean=git-lfs clean -- %f
file:C:/Program Files/Git/etc/gitconfig filter.lfs.smudge=git-lfs smudge -- %f
file:C:/Program Files/Git/etc/gitconfig filter.lfs.process=git-lfs filter-process
file:C:/Program Files/Git/etc/gitconfig filter.lfs.required=true
file:C:/Program Files/Git/etc/gitconfig http.sslbackend=schannel
file:C:/Program Files/Git/etc/gitconfig core.autocrlf=true
file:C:/Program Files/Git/etc/gitconfig core.fscache=true
file:C:/Program Files/Git/etc/gitconfig core.symlinks=false
file:C:/Program Files/Git/etc/gitconfig pull.rebase=false
file:C:/Program Files/Git/etc/gitconfig credential.helper=manager
file:C:/Program Files/Git/etc/gitconfig credential.https://dev.azure.com.usehttppath=true
file:C:/Program Files/Git/etc/gitconfig init.defaultbranch=master
file:C:/Users/Siu13/.gitconfig user.email=siu1310257@gmail.com
file:C:/Users/Siu13/.gitconfig user.name=Nathan
file:.git/config      core.repositoryformatversion=0
file:.git/config      core.filemode=false
file:.git/config      core.bare=false
```

```
file:.git/config    core.logallrefupdates=true
file:.git/config    core.symlinks=false
file:.git/config    core.ignorecase=true
file:.git/config
remote.origin.url=https://github.com/Slime917/1620-Fundamental-Web-Dev.git
file:.git/config    remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
file:.git/config    branch.master.remote=origin
file:.git/config    branch.master.merge=refs/heads/master
file:.git/config    branch.master.vscode-merge-base=origin/master
file:.git/config    branch.test.vscode-merge-base=origin/master
file:.git/config    branch.demo-branch.vscode-merge-base=origin/master
```

B) Initialize a Repository

6) Create a new folder called dev-notes and initialize it as a Git repository.

Command(s) — Answer:

```
mkdir dev-notes
cd dev-notes
git init
```

7) Create three files (index.html, styles.css, tips.txt) and stage everything for commit. (Hint: Use “echo” for creating the files)

Command(s) — Answer:

```
"<!DOCTYPE html>" | Out-File index.html
```

```
"body { }" | Out-File style.css
```

```
"Some notes" | Out-File tips.txt
```

```
git add .
```

8) Commit with the message: Initial commit: scaffold site (index, styles, tips).

Command(s) — Answer:

```
git commit -m "Initial commit: scaffold site (index, styles, tips)."
```

9) Verify that the .git directory exists and list tracked files.

Command(s) — Answer:

```
ls -Force  
git ls-files
```

Expected effect / output — Answer:

```
.git  
index.html  
styles.css  
tips.txt
```

C) Status, Add, and .gitignore

10) Show the working tree status.

Command(s) — Answer:

```
git status
```

Expected effect / output — Answer:

Shows modified/added files and staged status

11) Create a .gitignore that ignores *.log files and a node_modules/ directory. Stage and commit it.

Command(s) — Answer:

```
"*.log`node_modules/" | Out-File .gitignore  
git add .gitignore  
git commit -m "Add .gitignore for logs and node_modules"
```

12) Prove that a file named debug.log is ignored by Git.

Command(s) — Answer:

```
"test" | Out-File debug.log  
git status
```

Expected effect / output — Answer:

Untracked files:

 debug.log

D) Commits & Messages

13) Modify index.html, stage only that file, and commit with the message: Add main page intro.

Command(s) — Answer:

"Updated intro" >> index.html

git add index.html

git commit -m "Add main page intro"

14) Modify styles.css, stage it, and commit with the message: Tweak typography and spacing.

Command(s) — Answer:

```
"Front-size:16px;" >> styles.css  
git add styles.css  
git commit -m "Tweak typography and spacing"
```

15) Amend the most recent commit to change its message to: Refine typography and spacing.

Command(s) — Answer:

```
git commit --amend -m "Refine typography and space"
```

E) Inspecting History & Diffs

16) Show a one-line-per-commit graph with decorations (all branches).

Command(s) — Answer:

```
git log --oneline --graph --decorate --all
```

Expected effect / output — Answer:

```
* c6e2625 (HEAD -> master, origin/master, origin/HEAD) another_update
* fe0b907 wk10_notes_update
* 7bf2900 amUpdateTest
* 6c0d812 update3
* 547a338 update2
* 913e980 (demo-branch) update
* 38ced28 test2
* 01c40f2 tsst
* 272a8ab (test) Inital commit
```

17) Show differences for unstaged changes in the working directory.

Command(s) — Answer:

```
git diff
```

Expected effect / output — Answer:

Shows unstaged changes

18) Show differences for staged changes (index vs HEAD).

Command(s) — Answer:

git diff --cached

Expected effect / output — Answer:

Shows staged changes waiting to be committed

19) Show details of a specific commit given its hash (message, author, patch).

Command(s) — Answer:

```
git show <commit-hash>
```

Expected effect / output — Answer:

Commit metadata and lines that were modified

F) Branching & Switching

20) Create and switch to a new branch named feature/dark-mode.

Command(s) — Answer:

```
git checkout -b feature/dark-mode
```

21) Switch back to main, then create and switch to a new branch named feature/new-tips.

Command(s) — Answer:

```
git checkout main
```

```
git checkout -b feature/new-tips
```

22) List all local branches and indicate the current branch.

Command(s) — Answer:

```
git branch
```

Expected effect / output — Answer:

```
feature/dark-mode
```

```
*feature/new-tips
```

```
main
```

H) Undoing Changes Safely

23) Discard unstaged changes to styles.css.

Command(s) — Answer:

```
git checkout – styles.css
```

24) Unstage a file named tips.md but keep its working-directory edits.

Command(s) — Answer:

```
git restore –staged tips.md
```

25) Create a new commit that undoes the effects of a specific commit (hash provided by you).

Command(s) — Answer:

```
git revert <commit-hash>
```

Expected effect / output — Answer:

Creates a new commit that undoes the original

I) Remotes, Push & Pull

26) Add a remote named origin that points to <https://github.com/yourname/dev-notes.git>.

Command(s) — Answer:

```
git remote add origin https://github.com/yourname/dev-note.git
```

27) Rename your current branch to main and push it to origin, setting upstream for future pushes.

Command(s) — Answer:

```
git branch -M main  
git push -u origin main
```

28) Push two feature branches (feature/dark-mode and feature/new-tips) and set upstream.

Command(s) — Answer:

```
git push -u origin feature/dark-mode
```

```
git push -u origin feature/new-tips
```

29) Pull the latest changes from the upstream tracking branch.

Command(s) — Answer:

```
git pull
```

J) Tags & Releases

30) Create an annotated tag v1.0.0 with the message "First stable release".

Command(s) — Answer:

```
git tag -a v1.0.0 -m "First stable release"
```


31) Push your tags to the remote.

Command(s) — Answer:

```
git push --tags
```

32) Show all tags in the repository.

Command(s) — Answer:

```
git tag
```

Expected effect / output — Answer:

```
v1.0.0
```